Informe final · Alpha Centauri · Rocket Landing

Proyecto final del curso de MLOps · Universidad Externado · noviembre de 2026 · equipo Alpha Centauri: María Jimena Castaño Albarracín (mjcastano29-tech), Isaías Céspedes Novoa (isaiasexternado-a11y) y Jorge Horacio Rojas Criollo (jhrojas02)

Web: https://alpha-centauri-five.vercel.app · API: https://alpha-centauri-api.onrender.com · MLflow: https://alpha-centauri-mlflow.onrender.com · Repositorio: https://github.com/mjcastano29-tech/Alpha-Centauri · Entrega: etiqueta `v1.1.1` (índice en entregables.md)

1. Qué construimos

Un agente de aprendizaje por refuerzo que aterriza un cohete en un mundo 3D con viento, temperatura y presión variables, combustible limitado y, en los niveles superiores, un obstáculo que obliga a decidir la ruta. Alrededor del agente, el sistema de operación que pide la guía: ambientes versionados con contrato y ficha, experimentos con hipótesis y decisión, modelos recuperables por huella, un pipeline que entrena, evalúa, decide y publica, una API que sirve el modelo recomendado de cada nivel, una web que permite al docente ver, comparar y evaluar, y un procedimiento de rollback comprobado en producción.

Pieza	Dónde	Cómo se comprueba
Ambientes Gymnasium (N1 0.1.1, N2 v1 0.4, N3 v1 0.5; retirados 0.2 y 0.3)	`rocket_env/`, fichas `docs/ambiente-v0.*.md`	`python -m pytest` (113 pruebas de contrato, modelos y API)
23 ejecuciones en 6 experimentos	`resultados/runs/`, `configs/`, `manifests/`, MLflow	`python cli.py verificar` (SHA-256 de modelos, datos y configuración)
Pipeline de 8 etapas (Prefect)	`pipelines/entrenar_publicar.py`, evidencia en `pipelines/ejecuciones/` (21 ejecuciones)	cada ejecución deja su JSON con las etapas, la decisión y el commit publicado
API FastAPI	`api/main.py`	`GET /version`, `GET /monitoreo`, `POST /evaluar`, `POST /sesiones` (misión en vivo paso a paso)

Pieza	Dónde	Cómo se comprueba
Web Vite + TypeScript + Three.js	`web/`	portada con el cohete (pantalla completa con F), experimentos, catálogo, incidente, monitoreo, viento en vivo (`#/vivo`: barra de viento mientras el modelo congelado pilota), landing
Catálogo versionado con recomendado y anterior por nivel	`catalogo.json` (v15, 15 cambios en el historial)	`python catalogo.py ver`; rollback con `restaurar`

2. Los niveles y cómo crece la complejidad

	N1 · misión base	N2 · pasarela	N3 · pasarela con viento fuerte
Ambiente	`RocketLanding3D-v0` 0.1.1	`RocketLanding3D-N2-v1` 0.4	`RocketLanding3D-N3-v1` 0.5
Qué añade	plataforma, viento 0–10 m/s, atmósfera, combustible; paso = 20 m, 50 pasos	una pasarela a 180 m cubre la plataforma; la única abertura (r 16 m) está desplazada 30 m: hay que entrar por ella y trasladarse por debajo; "colisión" −100	viento 0–15 m/s y ráfagas dobles sobre el mismo mundo (misma familia de modelos)
Observación · acción	16 · 5	20 · 5 (familia nueva)	20 · 5
Dificultad medida	—	el recomendado de N1 a ciegas: 99 % → 5 % (190 colisiones)	el recomendado de N2 sin reentrenar: 100 % → 93,5 %
Recomendado	`ppo-v2-v0.1-s1`: 99 % (72 % con viento)	`ppo-pasarela-transfer-1m5-v0.4-s2`: 100 % (68 % con viento)	`ppo-pasarela-viento-transfer-1m5-desde-s2-v0.5-s0`: 97,5 % (85,5 % con viento)
Tres semillas del finalista	97, 99, 97,5 % (97,8 ± 1,0)	91,5, 97,5, 100 % (96,3 ± 4,4)	94,5, 94, 92,5 % (93,7 ± 1,0)
Test reservado (una sola vez)	97,5 % (viento 72,5 %)	99,0 % (viento 58,5 %)	97,0 % (viento 83,5 %)

La primera versión del nivel 2 fueron tres torres alrededor de la plataforma (ambiente 0.2, con su nivel 3 0.3). El equipo las sustituyó por la pasarela el 6 de noviembre porque obliga a una decisión más clara (entrar por la abertura y volver al centro) y se ve mejor en el replay. Los modelos de torres siguen en el catálogo como históricos y se pueden evaluar.

3. La cadena de hipótesis

Cada configuración tiene en `configs/<run_id>.json` su hipótesis, el cambio frente a su baseline, el presupuesto y la decisión con fecha; las mismas etiquetas están en MLflow. Los rechazos se conservan. Resumen por experimento (EXPERIMENTOS.md tiene cada uno con sus variables):

1. **Q-learning en N1** (`base-v0.1-s0`, 78 %): referencia aprendida, con dos tablas pequeñas y el empuje expresado respecto al peso.
2. **PPO en N1**: la acción cruda no aprende (0 %); la acción relativa v1 frena pero llega inclinada (0 %); la **v2** acota la inclinación al límite de aterrizaje y aprende (97 %). Tres semillas: 97,8 ± 1,0 %. La semilla 1 (99 %) es el recomendado por el criterio del pipeline.
3. **N2 con torres**: desde cero, 8 %; con transferencia de pesos desde N1, 77 % con 600 000 pasos y 86 % con 1,5 M; tres semillas 88,3 ± 4,5 %.
4. **N3 con torres**: incidente (−11 puntos sin reentrenar) e intervención por el pipeline: 92 % partiendo del nuevo recomendado de N2; tres semillas 89,3 ± 4,2 %. Rollback demostrado en producción (catálogos v8 → v9 → v10).
5. **N2 con pasarela**: el primer diseño (abertura fija en la observación, transferencia simple) no aprendía (2 % a los 320 000 pasos). Con la observación "objetivo de la fase" (la abertura por encima de la pasarela, la plataforma por debajo) y la herencia de los pesos de posición, la política de partida ya aterriza el 35 % sin entrenar, y tras 1,5 M de pasos 96,3 ± 4,4 % en tres semillas.
6. **N3 con pasarela**: incidente para las tres semillas (−27, −12,5 y −6,5 puntos) e intervención: la receta da 93,7 ± 1,0 %; partiendo del nuevo recomendado de N2, 97,5 % y 85,5 % con viento (24 puntos más que sin reentrenar).

Lo que aprendimos: la parametrización de la acción decide si PPO aprende; la transferencia desde el nivel anterior es lo que hace abordables los niveles 2 y 3 con 1,5 M de pasos; cuando el obstáculo cambia qué significa "objetivo", hay que dárselo al agente en la observación. Y la variabilidad entre semillas (hasta 4 puntos) es la razón de reportar medias y de que el pipeline conserve el recomendado y el anterior.

4. Pipeline, despliegue y monitoreo

`pipelines/entrenar_publicar.py` (Prefect 3) ejecuta ocho etapas con los mismos comandos que usamos a mano: validar el ambiente, probar el contrato, entrenar, evaluar en la suite v1 (200 épocas normales y 200 con viento de 15 m/s), registrar (configs, manifiesto de la sección 11 con commit completo y SHA-256, MLflow, huellas), decidir, publicar el catálogo con commit y push, y comprobar que la API pública sirve el commit y el catálogo esperados con una evaluación corta. Criterio de aceptación, fijado antes de entrenar: un candidato sustituye al recomendado de su nivel si gana al menos 2 puntos de tasa, no pierde más de 5 con viento y no tiene más colisiones; en un nivel nuevo se compara con el modelo del nivel anterior jugando en él (`incidente.py`). 18 ejecuciones del pipeline (9 aceptadas, 7 rechazadas, 2 sin decisión) y 3 comprobaciones de rollback dejaron su evidencia versionada.

Despliegue: push a `main` → Render reconstruye la API y MLflow, Vercel la web. `GET /version` dice qué commit y qué catálogo se sirven; `GET /monitoreo` y la vista `#/monitoreo` muestran el servicio, el modelo que atiende cada nivel y su comportamiento, las últimas decisiones del pipeline y los rollbacks, la latencia de las evaluaciones servidas y alertas (API recién despierta, recomendado frágil con viento, ejecución entrenada con cambios sin commit).

5. Incidente, intervención y rollback

- **Incidente controlado**: cambiar las condiciones de operación (viento 0–15 m/s, ráfagas dobles). Medido antes y después para cada modelo con `incidente.py` (9 medidas en `resultados/incidentes/`, vista `#/incidente`).
- **Intervención**: reentrenar en el nivel 3 partiendo de los pesos del modelo degradado. Se aceptó o rechazó con el criterio del pipeline; en la pasarela recupera 24 puntos con viento máximo.
- **Rollback**: `python catalogo.py restaurar --nivel N3` devuelve el anterior; se demostró en producción con las torres (catálogo v9) y se volvió al recomendado (v10), comprobando la API cada vez (`comprobar_api.py`, evidencia en `pipelines/ejecuciones/rollback-n3-*.json`). El catálogo actual guarda el anterior de cada nivel para repetirlo.

6. Límites y decisiones honestas

- 8 de las 23 ejecuciones se entrenaron con cambios sin commit (marca `*`); en cada caso el archivo sin confirmar era documentación o el código del propio nivel, y la ficha lo documenta. El pipeline avisa en la etapa 1.
- El recomendado de cada nivel es la mejor de tres semillas según el criterio; la cifra honesta de la configuración es la media (97,8, 96,3 y 93,7 %). El test reservado confirma que la selección no sobreajustó a la suite v1 (diferencias de −1,5, −1 y −0,5 puntos).
- El plan gratuito de Render duerme la API y MLflow tras 15 minutos: la primera petición tarda cerca de un minuto.
- `mlflow.db` vive en git: dos personas no pueden ejecutar el pipeline a la vez (ocurrió una vez; se descartó la copia duplicada).
- La pasarela se dibuja translúcida y se vuelve casi transparente cuando queda entre la cámara y el cohete; el replay es una interpolación de trayectorias grabadas, no una simulación en el navegador.

7. Contribuciones

Según el historial de commits y las pull requests del repositorio:

- **María Jimena Castaño Albarracín** (mjcastano29-tech, con Claude Code como asistente): ambiente y física, agentes Q-learning y PPO, experimentos 1 y 2, nivel 2 (torres y pasarela), versionado (manifiestos, configs, suites, MLflow), API, web

(catálogo, experimentos, replay 3D, monitoreo), pipeline Prefect, diagnósticos e informe.

- **Isaías Céspedes Novoa** (isaiasexternado-a11y): escena 3D cinematográfica original, pruebas de contrato y de modelos, huellas SHA-256 y `cli.py verificar`, dependencias fijadas, landing de presentación, consola de misión y sonido del replay.
- **Jorge Horacio Rojas Criollo** (jhrojas02): nivel 3 (ambientes 0.3 y 0.5), `incidente.py` e intervención, rollback en producción, `comprobar_api.py`, tres semillas de los finalistas de N3 y de la pasarela, MLflow desde cualquier máquina.

8. Cómo reproducir

```
git clone https://github.com/mjcastano29-tech/Alpha-Centauri && cd Alpha-Centauri
python -m venv .venv && .venv/bin/pip install -r requirements.txt
python -m pytest                        # 113 pruebas
python cli.py verificar                 # huellas de las 23 ejecuciones
python cli.py evaluar --run ppo-pasarela-viento-transfer-1m5-desde-s2-v0.5-s0 --viento 15
uvicorn api.main:app --port 8000        # API local; web: cd web && npm install && npm run dev
python entregables.py                   # regenera los PDF de entregables/
```